

NCTU-EE ICLAB – Fall 2025

Lab11 Exercise:

Design: Geometric Transform Engine (GTE)

Data Preparation

- Extract the data from TA's directory:
`% tar xvf ~iclabTA01/Lab11_2025F.tar`

Design Description

In modern image and video processing systems, spatial manipulation of pixel data plays a crucial role in enabling efficient computation, data compression, and visual feature extraction. Operations that modify the geometric structure or access pattern of an image, such as mirroring, rotation, shifting, and reordering, are frequently employed in hardware-accelerated architectures.

For example, mirror and rotation operations are widely used in image enhancement, data augmentation for deep learning, and alignment correction. Shifting operations are often required in motion estimation and local window-based filtering, where image boundaries must be handled carefully through padding or wrapping schemes. Meanwhile, data reordering techniques, such as zig-zag scanning in JPEG compression or Morton (Z-order) mapping in memory organization, play a critical role in improving spatial locality, thereby reducing memory bandwidth and improving cache efficiency in hardware and embedded systems.

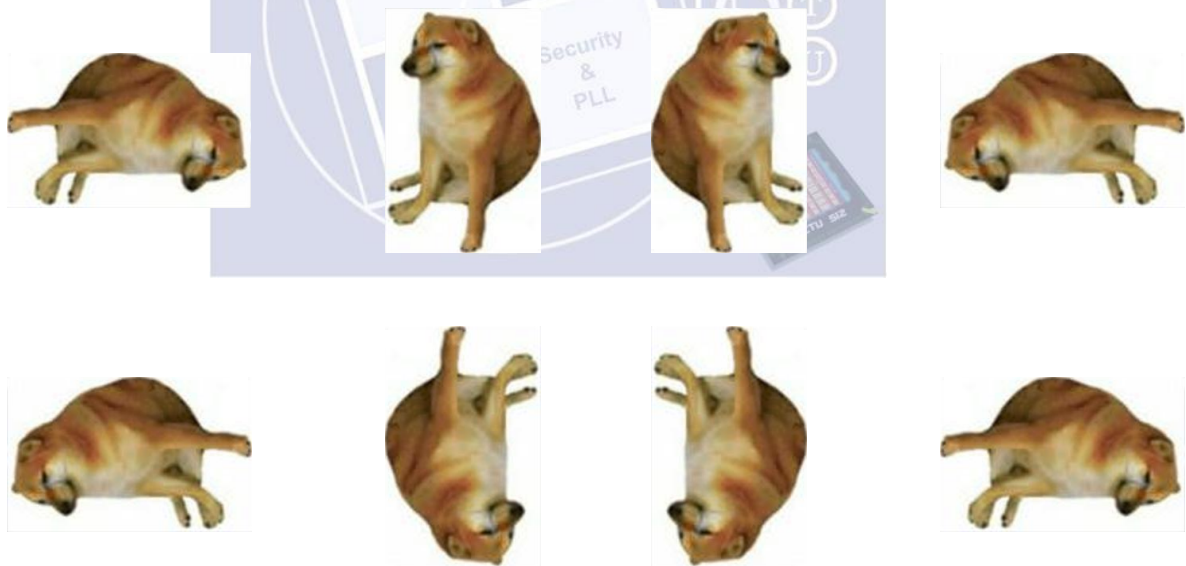


Figure 1. Example of Geometric image processing

In this exercise, you need to build a design to perform **Geometric Transform Engine (GTE)** with several commands. For the images require large space to store, you are asked to use **memory generated by TA** for finishing this lab.

- Operation

There are 15 possible operations will be given:

- Mirror

- Mirror along the X-axis (MX)

The **MX operation** performs a **vertical flip** of the image along the X-axis. Pixels are mirrored **top-to-bottom**, meaning that the first row of the image becomes the last row, the second row becomes the second-to-last, and so on.

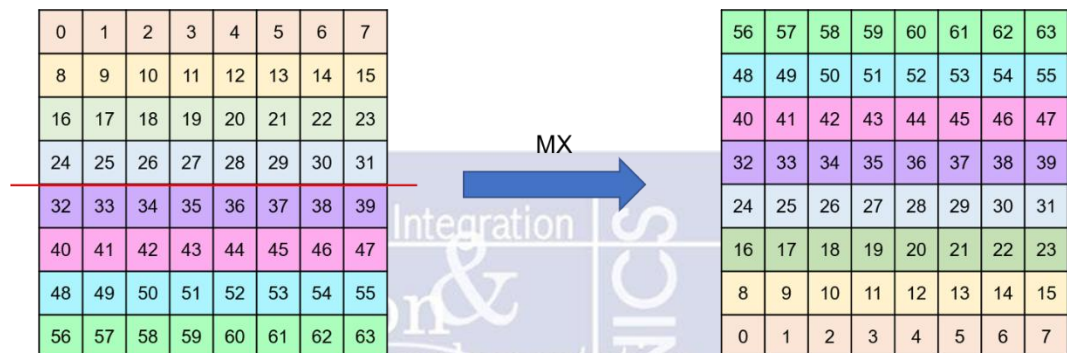


Figure 2. MX operation for 8x8 images

- Mirror along the Y-axis (MY)

The **MY operation** performs a **horizontal flip** of the image along the Y-axis. Pixels are mirrored **left-to-right**, so that the first column becomes the last, the second column becomes the second-to-last, and so on.

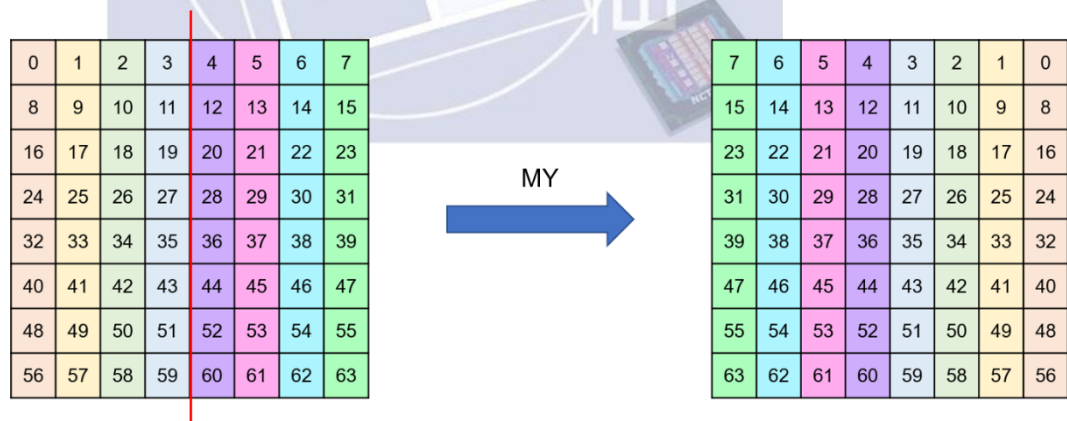


Figure 3. MY operation for 8x8 images

- Transpose (TRP)

The Transpose operation reflects the image along its **main diagonal**, which extends

from the **top-left** to the **bottom-right** corner. Pixels are mirrored **diagonally**, with the top-right region of the image mapped to the bottom-left region, and vice versa.

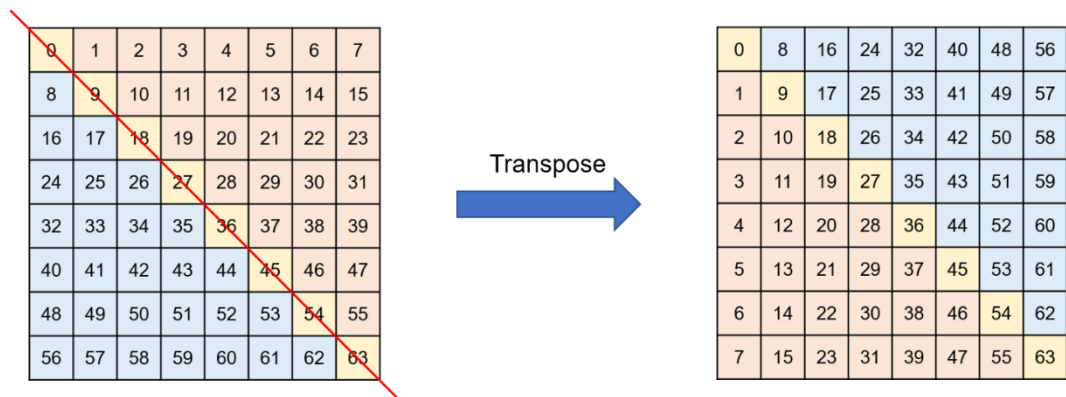


Figure 4. Transpose operation for 8x8 images

■ Secondary Transpose (STRP)

The Secondary Transpose operation reflects the image along its **secondary diagonal**, which extends from the **top-right** to the **bottom-left** corner. Pixels are mirrored **diagonally in the opposite direction**, with the top-left region of the image mapped to the bottom-right region, and vice versa.

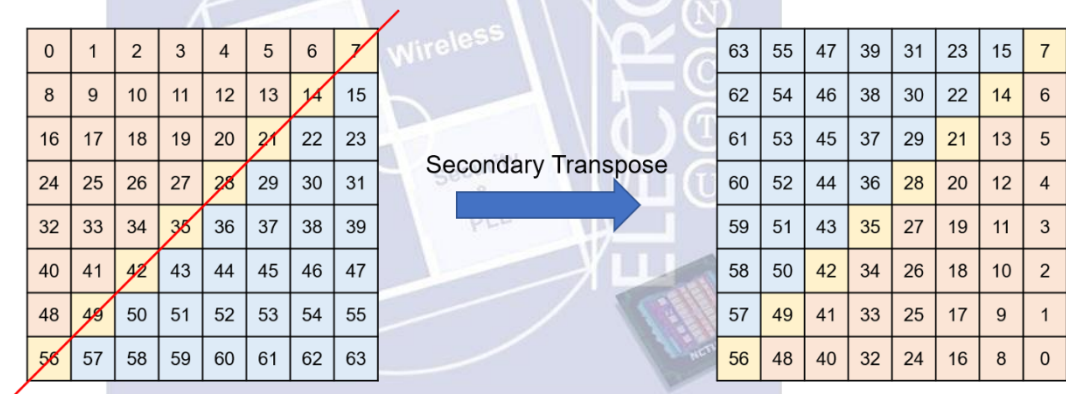


Figure 5. Secondary Transpose operation for 8x8 images

➤ Rotation

■ Rotate 90° clockwise (R90)

The R90 operation rotates the image 90 degrees clockwise around its center. Pixels from the top row move to the rightmost column, and the leftmost column becomes the top row.

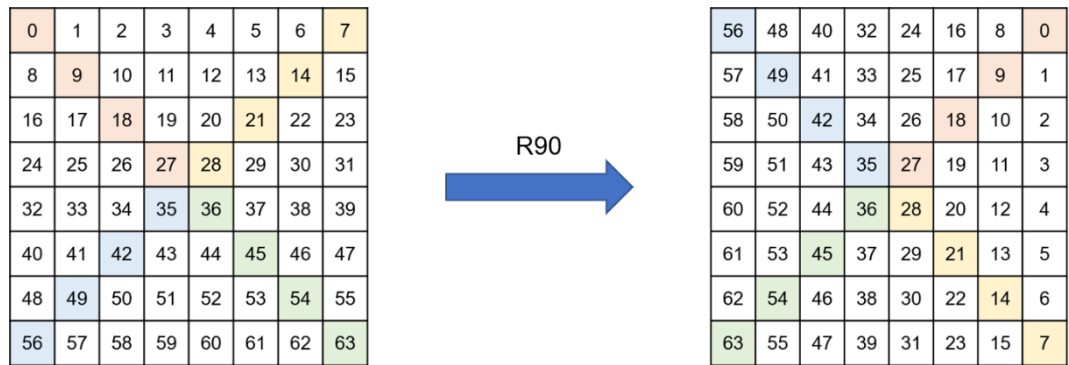


Figure 6. R90 operation for 8x8 images

■ Rotate 180° (R180)

The R180 operation rotates the image 180 degrees around its center. Pixels are inverted both horizontally and vertically, meaning the image appears upside down.

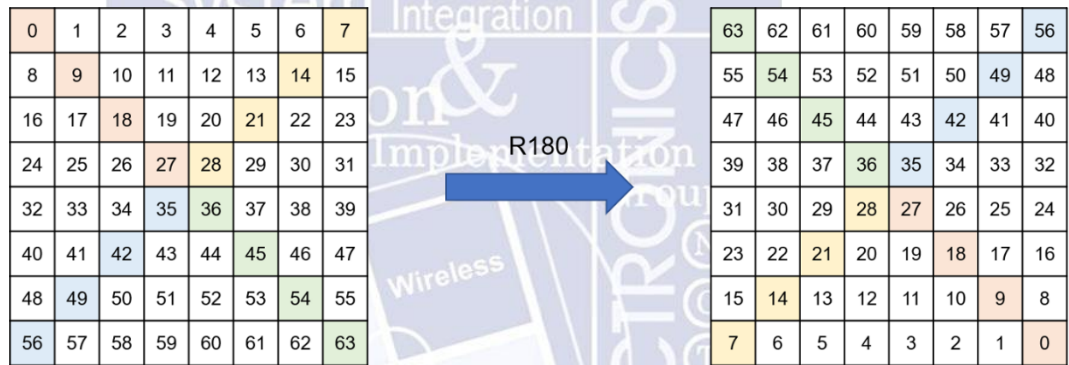


Figure 7. R180 operation for 8x8 images

■ Rotate 270° clockwise (R270)

The R270 operation rotates the image 270 degrees clockwise (or equivalently 90 degrees counterclockwise). Pixels from the top row move to the leftmost column, and the rightmost column becomes the top row.

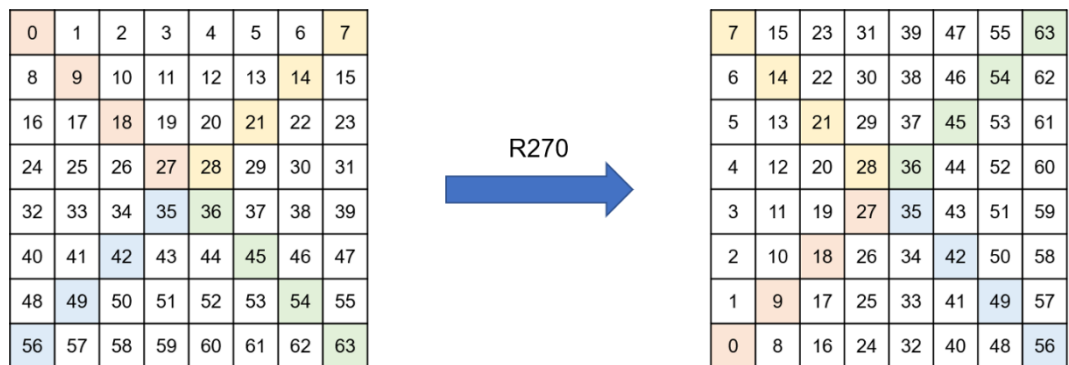


Figure 8. R270 operation for 8x8 images

➤ Shift (mirror padding)

■ Right shift (RS)

The image is shifted 5 pixels to the right. Pixels originally on the right edge move outside the visible region, while empty spaces on the left are filled using a mirrored reflection of the original leftmost columns.

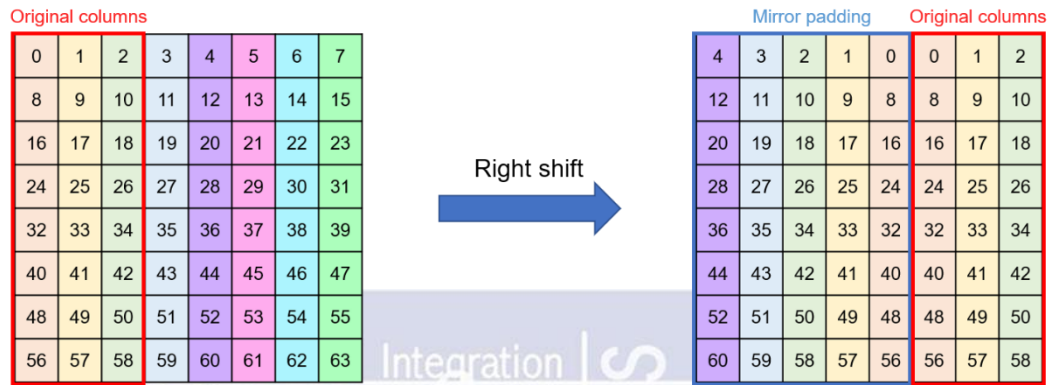


Figure 9. Right shift operation for 8x8 images

■ Left shift (LS)

The image is shifted 5 pixels to the left. Pixels originally on the left edge move outside the visible region, while empty spaces on the right are filled using a mirrored reflection of the original rightmost columns.

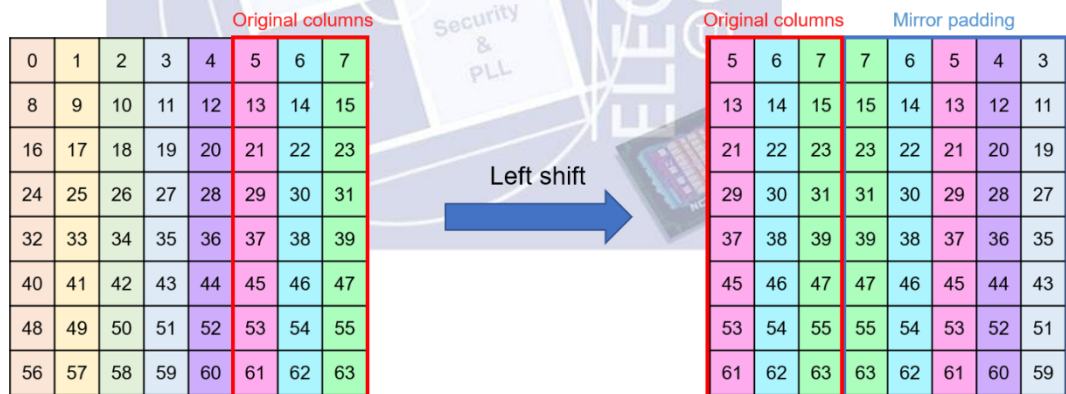


Figure 10. Left shift operation for 8x8 images

■ Up shift (US)

The image is shifted 5 pixels upward. Pixels originally on the top edge move outside the visible region, while empty spaces on the bottom are filled using a mirrored reflection of the original top rows.

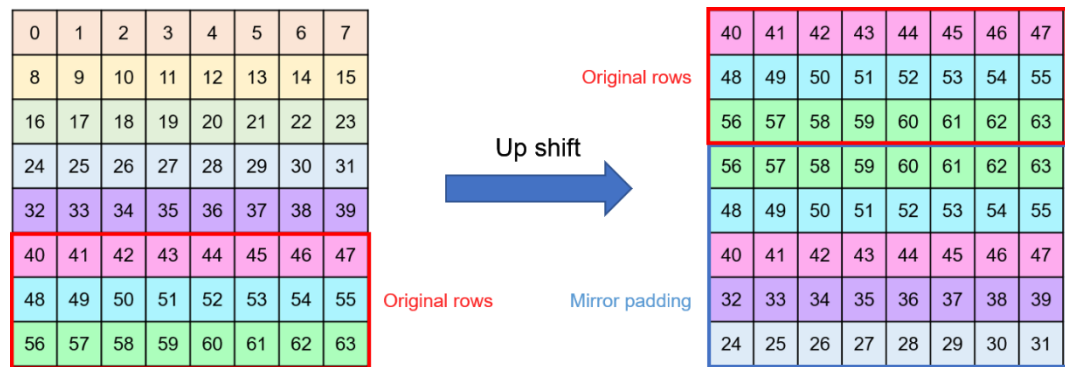


Figure 11. Up shift operation for 8x8 images

■ Down shift (DS)

The image is shifted 5 pixels downward. Pixels originally on the bottom edge move outside the visible region, while empty spaces on the top are filled using a mirrored reflection of the original bottom rows.

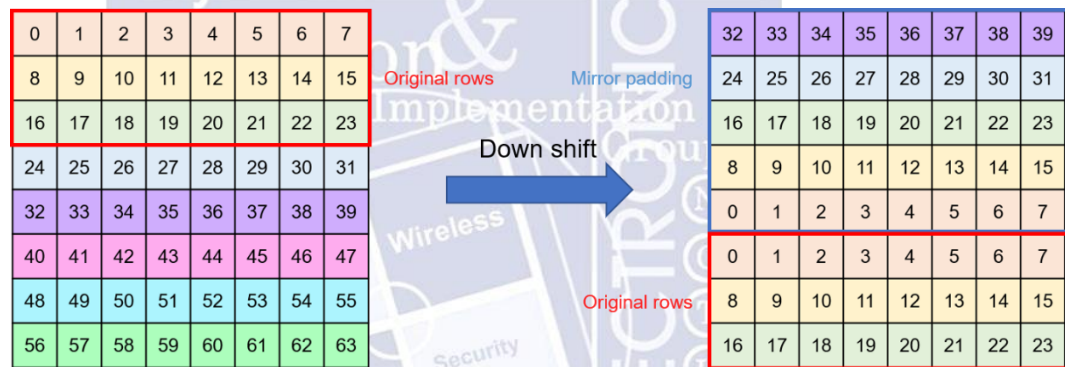


Figure 12. Down shift operation for 8x8 images

➤ Reorder

■ 4x4 Zig-zag (ZZ4)

The image is divided into multiple 4x4 sub-blocks. Each sub-block is reordered following a zig-zag scanning pattern that traverses diagonally from the top-left to the bottom-right.

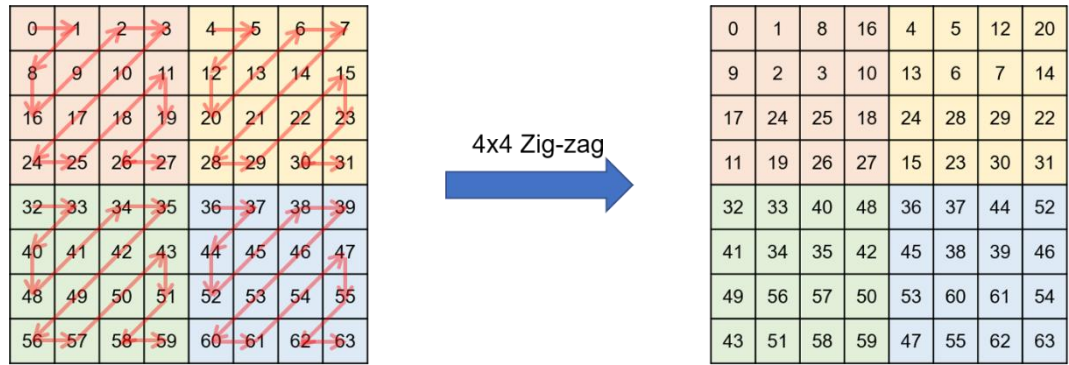


Figure 13. 4x4 Zig-zag operation for 8x8 images

■ 8x8 Zig-zag (ZZ8)

The image is divided into multiple 8×8 sub-blocks. Each sub-block is scanned in a zig-zag pattern from the top-left to the bottom-right, reordering the pixel positions according to diagonal progression.

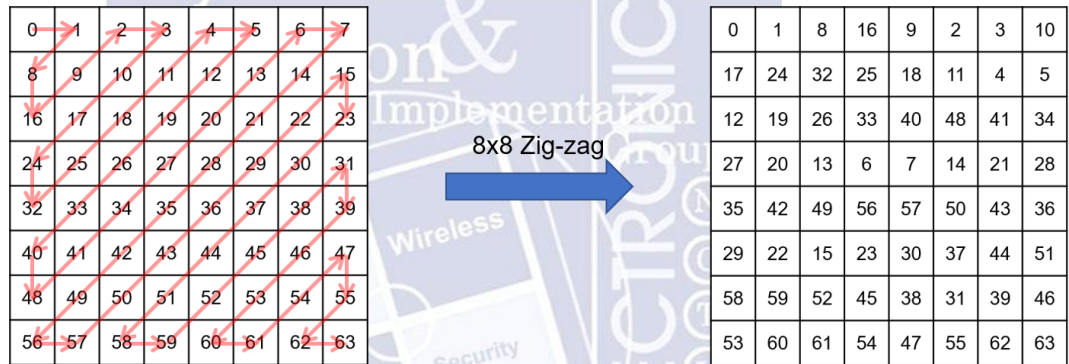


Figure 14. 8x8 Zig-zag operation for 8x8 images

■ 4x4 Morton order (MO4)

The image is divided into multiple 4×4 sub-blocks. Each sub-block is reordered following a Morton order pattern, which interleaves row and column indices to produce a recursive “Z”-shaped traversal.

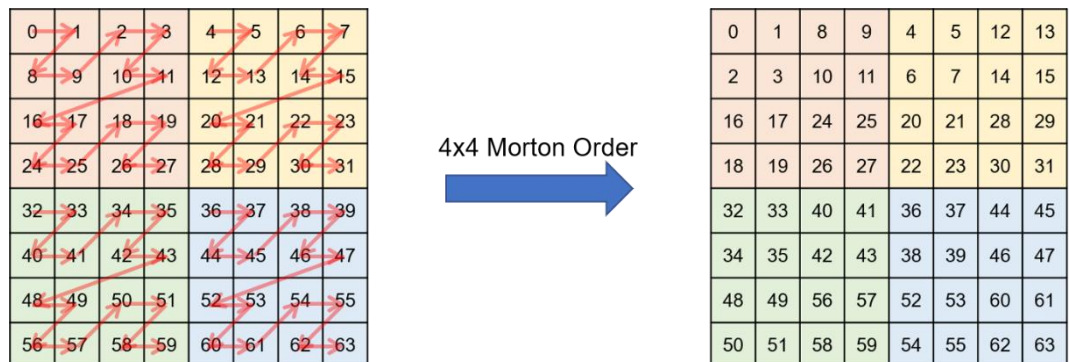


Figure 15. 4x4 Morton Order operation for 8x8 images

■ 8x8 Morton order (MO8)

The image is divided into multiple 8×8 sub-blocks. Each sub-block follows a Z-order traversal, visiting pixels in a pattern that alternates between horizontal and vertical directions to form a fractal-like “Z” path.

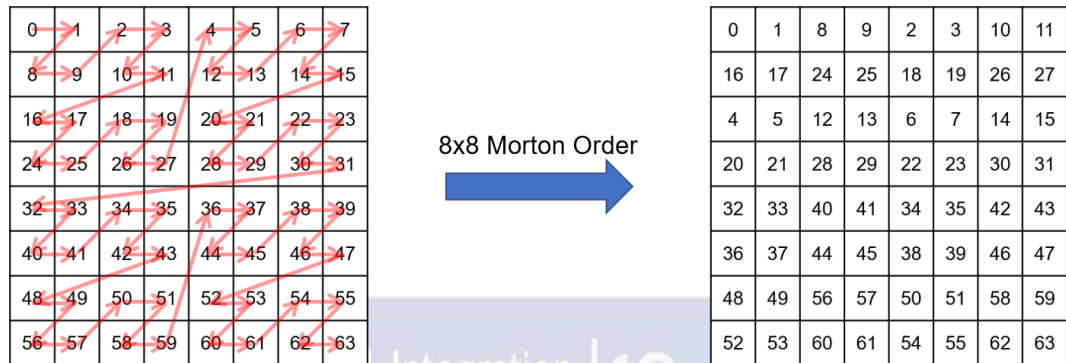


Figure 16. 8x8 Morton Order operation for 8x8 images

● Rules of input signals

In this exercise, you will receive the data of 128 images, numbered from 0 to 127, provided **sequentially** at the beginning of simulation.

➤ Input Image

Each image has a **fixed size of 16×16 pixels**. Each pixel represents the grayscale component and is stored as an **8-bit unsigned value**. The 16×16 frame is provided in **raster-scan order** from image 0 to image 127.

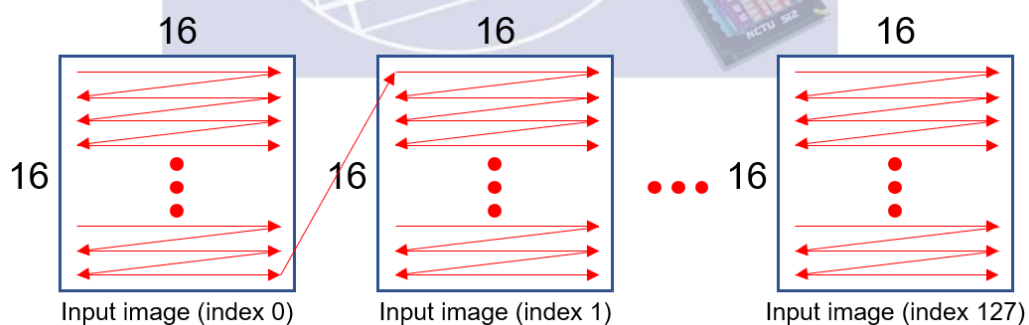


Figure 17. Input order: raster scan order for each 16x16 image

➤ SRAMs for 128 images

You are asked to use **eight SRAMs (three types)** generated by TA. The following is the spec for each type of SRAM:

- SUMA180_4096X8X1BM4: It is a single-port SRAM that can store 4096 words of 8 bits each. In this lab, you will use four SRAMs of this type to store images: MEM0 stores image 0~15, MEM1 stores image 16~31, MEM2 stores image 32~47 and MEM3 stores image 48~63, respectively.

address	Data
0	image_0[0][0]
1	image_0[0][1]
⋮	
255	image_0[15][15]
256	image_1[0][0]
257	image_1[0][1]
⋮	
4094	image_15[15][14]
4095	image_15[15][15]

MEM0 (image 0~15)

Figure 18. Stored data representation of MEM0

- SUMA180_2048X16X1BM1: It is a single-port SRAM that can store 2048 words of 16 bits each. In this lab, you will use two SRAMs of this type to store images: MEM4 stores image 64~79 and MEM5 stores image 80~95, respectively.

address	Data
0	{image_64[0][0], image_64[0][1]}
1	{image_64[0][2], image_64[0][3]}
⋮	
127	{image_64[15][14], image_64[15][15]}
128	{image_65[0][0], image_65[0][1]}
129	{image_65[0][2], image_65[0][3]}
⋮	
2046	{image_79[15][12], image_79[15][13]}
2047	{image_79[15][14], image_79[15][15]}

MEM4 (image 64~79)

Figure 19. Stored data representation of MEM4

- SUMA180_1024X32X1BM2: It is a single-port SRAM that can store 1024 words of 32 bits each. In this lab, you will use two SRAMs of this type: MEM6 stores image 96~111 and MEM7 stores image 112~127, respectively.

address	Data
0	{image_96[0][0], image_96[0][1], image_96[0][2], image_96[0][3]}
1	{image_96[0][4], image_96[0][5], image_96[0][6], image_96[0][7]}
⋮	
63	{image_96[15][12], image_96[15][13], image_96[15][14], image_96[15][15]}
64	{image_97[0][0], image_97[0][1], image_97[0][2], image_97[0][3]}
65	{image_97[0][4], image_97[0][5], image_97[0][6], image_97[0][7]}
⋮	
1022	{image_111[15][8], image_111[15][9], image_111[15][10], image_111[15][11]}
1023	{image_111[15][12], image_111[15][13], image_111[15][14], image_111[15][15]}

MEM6 (image 96~111)

Figure 20. Stored data representation of MEM6

After that, you will receive an 18-bit command if **busy** is tied low for each pattern. The following are some rules about the command:

➤ Command

The following is the format for the command

MSB	Format – (18 bits)			LSB
opcode (2-bit)	funct (2-bit)	ms (7-bit)	md (7-bit)	

Memory source index (ms) and Memory destination index (md) represent the index of source images and destination images, respectively. Since the command takes 7 bits to store the index, it means we have 128 images, from image 0 to image 127. For example, ms = 7'd25 means the 25th image stored in SRAM is the location of the source image, while md = 7'd102 means the 102th image stored in SRAM is the location of the destination image.

opcode and **funct** will determine which operation mentioned above should be executed. The details are shown as below:

Command Binary Encode	Operation Action	Meaning
00-00-ms-md	image[md] = MX(image[ms])	Mirror along X-axis
00-01-ms-md	image[md] = MY(image[ms])	Mirror along Y-axis
00-10-ms-md	image[md] = TRP(image[ms])	Transpose
00-11-ms-md	image[md] = STRP(image[ms])	Secondary Transpose
01-00-ms-md	image[md] = R90(image[ms])	Rotate 90° clockwise
01-01-ms-md	image[md] = R180(image[ms])	Rotate 180°
01-10-ms-md	image[md] = R270(image[ms])	Rotate 270° clockwise
10-00-ms-md	image[md] = RS(image[ms])	Right shift
10-01-ms-md	image[md] = LS(image[ms])	Left shift
10-10-ms-md	image[md] = US(image[ms])	Up shift
10-11-ms-md	image[md] = DS(image[ms])	Down shift
11-00-ms-md	image[md] = ZZ4(image[ms])	4x4 Zig-zag

11-01-ms-md	image[md] = ZZ8(image[ms])	8x8 Zig-zag
11-10-ms-md	image[md] = MO4(image[ms])	4x4 Morton Order
11-11-ms-md	image[md] = MO8(image[ms])	8x8 Morton Order

- Rules of operation execution

After receiving the command, **GTE** should decode the command, load the source image from SRAM, implement the specified operation mentioned above and store the destination image to SRAM. For example, if command is 00-00-000000-111111, **GTE** should fetch the 0th image of SRAM first, do MX operation and eventually store the result image to the 127th image of SRAM.

- Rules of output result

When finishing a command, you need to tie low **busy** to obtain the next command. **When busy is tied low, pattern will only fetch the image[md] stored in SRAM and check answer.** (You should check the fetch method in PATTERN.v)

I/O Description

Input	Bit Width	Definition and Description
clk	1	Clock
rst_n	1	Asynchronous active-low reset
in_valid_data	1	High when data is valid.
data	8	The grayscale value of the input image is an 8-bit unsigned value (0~255), sent in raster-scan order continuously when in_valid_data is high until 128 images are already.
in_valid_cmd	1	High when cmd is valid.
cmd	18	It will be given in one cycle when in_valid_cmd is high. Format: {opcode, funct, ms, mt} = {2-bit, 2-bit, 7-bit, 7-bit}
Output	Bit Width	Definition and Description
busy	1	Pull high when chip is busy. It should be low for one cycle whenever you finished a command. TA will check the values of destination images stored in SRAM when your busy is low. (please check how to check the images in PATTERN.v)

Specification

Top module

- Top module name: **GTE** (file name: **GTE.v**)
- You can adjust your clock period for 03_GATE and 06_POST by yourself, respectively, but the maximum period of both is **20.0 ns**. For example, cycle time for 03_GATE can be 15.5, while cycle time for 06_POST can be 20.0.
- The precision of clock is 0.1, for example, 19.5 is allowed but 19.55 is not.

4. The execution latency is limited to **5000 cycles**. The latency is the clock cycles between the falling edge of the last cycle of **in_valid_cmd** and the negative edge of the first cycle of **busy**.
5. The total cell area should not larger than **5,000,000 μm^2** .
6. In this lab, you **must use all the memories defined by TA**. We will check it at **GTE.area in 02_SYN/Report**. The area of **Macro/Black Box** must not be less than **2,100,000 μm^2** . The example is shown in following figure.

```

Combinational area:.....1821995.696653
Buf/Inv area:.....111973.280126
Noncombinational area:.....343750.185371
Macro/Black Box area:.....214305.703125
Net Interconnect area:.....undefined (No wire load specified)

Total cell area:.....2380051.585150

```

7. If any port of memory is connected with mismatched width, the memory will not be synthesized and you will get an error message. Even though the design may still pass gate level simulation, this situation will be regarded as a synthesis fail. In this case, the memory area will be 0 in **GTE.area**. We will check it at **syn.log** and **GTE.area**.

clock & rst_n

8. It is **asynchronous reset** and **active-low** architecture. If you use synchronous reset (reset after clock starting) in your design, you may fail to reset signals that should be reset after the reset signal is asserted.
9. The reset signal (**rst_n**) will be given only once at the beginning of simulation. **The output signal busy should be reset to 1 after the reset signal is asserted**. The reset duration will last for 3 times the cycle time.

in_valid & data

10. All input signals are synchronized at negative edge of the clock.
11. **in_valid_data** will come after reset.
12. The input of 128 **images with size of 16x16** is delivered in **raster scan order** for **32768 cycles continuously** when **in_valid_data** is tied high. **When all elements of 128 images are delivered completely, it is tied to unknown state**, and **in_valid_data** will also be tied low.
13. The image data will only be delivered in the beginning of simulation. The first command will come in **2~4 negative edge of clock** after **in_valid_data** falls.

in_valid_cmd & cmd

14. The input of **cmd** is delivered for **only one cycle** when **in_valid_cmd** is tied high. **When the content of cmd is delivered completely, it will be tied to unknown state**.

15. **GTE** will receive commands until all of them are delivered from pattern.

busy

16. All output signals should be synchronized at clock positive edge.

17. The TA's pattern will capture your output for checking at **clock negative edge**.

18. **busy** cannot overlap with **in_valid_data** and **in_valid_cmd** at any time.

19. **busy** is limited to be tied **low** only when **GTE** is not busy.

20. The next command will come in **2~4 negative edge of clock after busy falls and pattern checks answer**.

Synthesis

21. The input delay is set to **0.5*(clock period)**.

22. The output delay is set to **0.5*(clock period)**, and the output loading is set to the **load of output pad: YA2GSD (about 0.024)**.

23. The input delay of **clk** and **rst_n** should be zero.

24. The synthesis time should be less than **4 hours**.

25. The synthesis result (syn.log) of data type cannot include any **latches** and **error**.

26. After synthesis, you can check **GTE.area** and **GTE.timing**. The area report is valid when the slack in the end of timing report should be **non-negative** and the result should be **MET**.

Gate-level simulation

27. The gate-level simulation cannot include any timing violations without the *notimingcheck* command.

28. **Error message about SRAM is not allowed** during both RTL and gate-level simulation

APR

29. Floorplanning

- Core size: Defined by user
- Core to IO boundary: Each side must be more than **100**
- Hard Macro placement: All hard macro should be **in CORE**

30. Powerplanning

- Core Ring
 - (i) Top & Bottom: metal layer must be **odd** (1,3,...) and width is **9**
 - (ii) Left & Right: metal layer must be **even** (2,4,...) and width is **9**
 - (iii) Each side must be **wire group, interleaving**, and at least **5 pairs**.
- Stripes
 - (i) Horizontal: metal layer must be **odd** (1,3,...) and width is at least **2**
 - (ii) Vertical: metal layer must be **even** (2,4,...) and width is at least **2**
 - (iii) The maximum distance between two stripes or stripe and edge should be less than **200**
- Power pads
 - (i) Core power pad: at least 4 pairs and well connected to power ring
 - (ii) IO power pad: at least 4 pairs.

31. Timing analysis results

- Timing slack: **No negative slacks** after **post-route setup/hold time analysis (including SI)**
- Design Rule Violation (DRV): The **DRV of (fanout, cap, tran)** should be **all 0** after **post-Route setup/hold time analysis (including SI)**

32. Design verification results

- Layout v.s. Schematic (LVS): **NO LVS violations** after “verify Connectivity”
- Design Rule Check (DRC): **NO DRC violations** after “verify DRC”
- IO Filler and core fillers should be added.

33. CHIP area should not larger than **10,000,000 μm^2** .

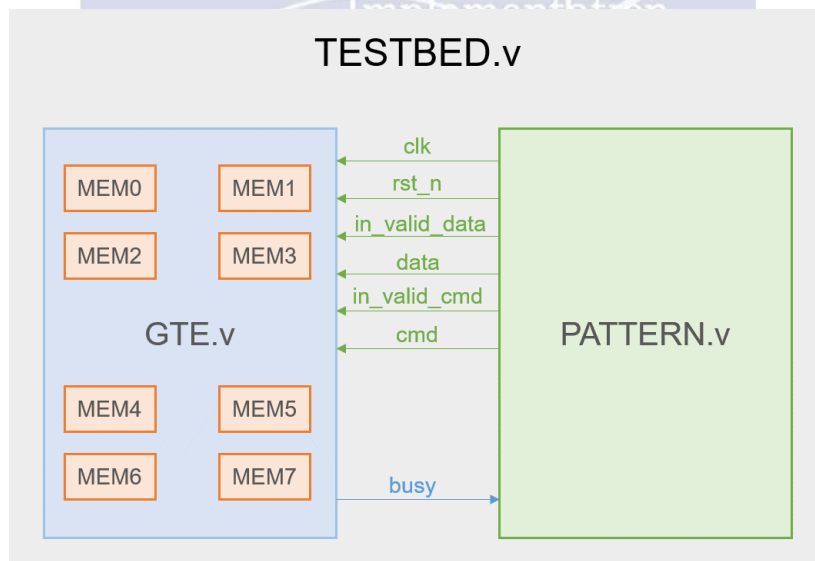
Post-layout simulation

- 34. The post-simulation cannot include any timing violations without the *notimingcheck* command.
- 35. **Error message about SRAM is not allowed** during post-layout simulation.
- 36. **The latency of post-layout simulation should be the same as that of gate-level simulation, or you will be considered fail in this lab.**

Supplement

37. In this lab, you are **NOT** allowed to use Designware IP.
38. Don't use any wire/reg/submodule/parameter name called **error**, **congratulation**, **latch** or **fail** otherwise you will fail the lab. Note: *** means any char in front of or behind the word. e.g: *error_note* is forbidden.
39. Don't write Chinese or other language comments in the file you sent.
40. Verilog command `//synopsys dc_script_begin`, `//synopsys dc_script_end` `//synopsys translate_off`, `//synopsys translate_on` are only allowed during the usage of including and setting designware IPs, other design compiler optimizations are forbidden.
41. Using the above commands are allowed, however any error messages during synthesis and simulation, regardless of the result will lead to failure in this lab.
42. Any form of display or printing information in verilog design is forbidden. You may use this methodology during debugging, but the file you turn in should not contain any coding that is not synthesizable.

Block Diagram



Grading Policy

The performance is determined by the area and latency of your chip. The less cost your design has, the higher grade you get.

- RTL/Gate-level/Post-level simulation, Synthesis and APR correctness: 70%
- Performance: 30% (06_POST)
 - **Chip area * Total latency**
 - Total latency = cycle time * latency

Note

- Please submit your design in Lab11/09_SUBMIT
 - 1st_demo deadline: 2025/12/01 (Mon.) 12:00:00
 - 2nd_demo deadline: 2025/12/03 (Wed.) 12:00:00 (score will be 30% off)
- Please upload the following files under 09_SUBMIT
 - 04_MEM_iclabXXX (Folder)

All memory-related files must be placed directly under this folder. Do not create any subdirectories (e.g., 04_MEM_iclabXXX/04_MEM) to store the files, or your lab will be considered failed. The submitted .lef file should be modified version.

Memory.v / Memory_WC.db / Memory_BC.lib / Memory_WC.lib / Memory.lef

- RTL design: GTE_iclabXXX.v
- filelist.f: filelist_iclabXXX.f
 - The memory path defined in filelist.f should be related path instead of absolute path.
 - No comment contexts are allowed in filelist.f
 - TA strongly recommends the following style to avoid any errors during demo.



```
TESTBED.v
../04_MEM/SUMA180_4096X8X1BM4.v
../04_MEM/SUMA180_2048X16X1BM1.v
../04_MEM/SUMA180_1024X32X1BM2.v
```

- APR file (The files will be fetched from 05_APR):

**CHIP_iclabXXX.sdc / CHIP_cts_iclabXXX.sdc / CHIP_iclabXXX.inn /
CHIP_iclabXXX.io / CHIP_iclabXXX.v / CHIP_iclabXXX.sdf /
CHIP_iclabXXX.inn.dat**

- Please make sure your file tree structure matches the figure below after you execute the 3 commands: 00_tar, 01_submit and 02_check in 09_SUBMIT sequentially. If your files violate the naming rules or directory structure, your lab will be marked as failed. You can use the command: “tree -L <layer>” to know your file tree structure.

```
/Lab11_2025F/09_SUBMIT]$ tree -L 2
```

```
-- Lab11_iclabTA07
-- 04_MEM_iclabTA07
-- CHIP_cts_iclabTA07.sdc
-- CHIP_iclabTA07.inn
-- CHIP_iclabTA07.inn.dat
-- CHIP_iclabTA07.io
-- CHIP_iclabTA07.sdc
-- CHIP_iclabTA07.sdf
-- CHIP_iclabTA07.v
-- GATE_15.5_iclabTA07.txt
-- GTE_iclabTA07.v
-- POST_18.5_iclabTA07.txt
-- fileList_iclabTA07.f
-- Lab11_iclabTA07.tar.gz
```

```
/Lab11_2025F/09_SUBMIT/Lab11_iclabTA07]$ tree -L 2
```

```
-- 04_MEM_iclabTA07
-- SUMA180_1024X32X1BM2.lef
-- SUMA180_1024X32X1BM2.v
-- SUMA180_1024X32X1BM2_BC.lib
-- SUMA180_1024X32X1BM2_wc.db
-- SUMA180_1024X32X1BM2_wc.lib
-- SUMA180_2048X16X1BM1.lef
-- SUMA180_2048X16X1BM1.v
-- SUMA180_2048X16X1BM1_BC.lib
-- SUMA180_2048X16X1BM1_wc.db
-- SUMA180_2048X16X1BM1_wc.lib
-- SUMA180_4096X8X1BM4.lef
-- SUMA180_4096X8X1BM4.v
-- SUMA180_4096X8X1BM4_BC.lib
-- SUMA180_4096X8X1BM4_wc.db
-- SUMA180_4096X8X1BM4_wc.lib
```

- Template folders and reference commands:

- 01_RTL/ (RTL simulation) `./01_run_vcs_rtl`
- 02_SYN/ (Synthesis) `./01_run_dc_shell`

- Check if there is any latch in your design in syn.log
- Check the timing of design in /Report/GTE.timing
- Check the area of design in /Report/GTE.area

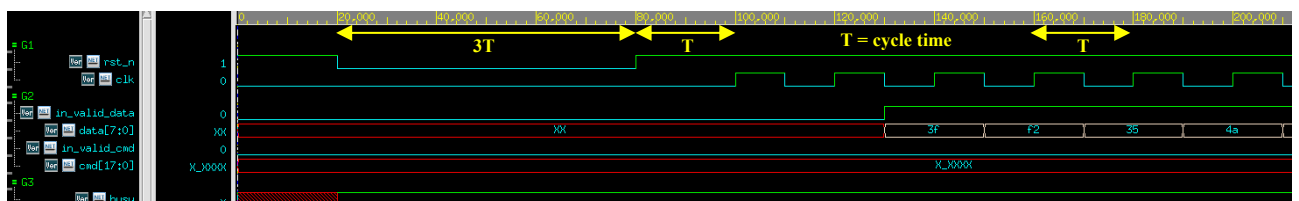
- 03_GATE/ (Gate-level simulation) `./01_run_vcs_gate`
- 04_MEM/ (Memory location)
- 05_APR/ (APR Folder) `./00_combine`
`./01_setenv`
- 06_POST/ (Post-level simulation) `./01_run_vcs_post`
- 09_SUBMIT/ (Submit your files) `./00_tar`

`./01_submit`

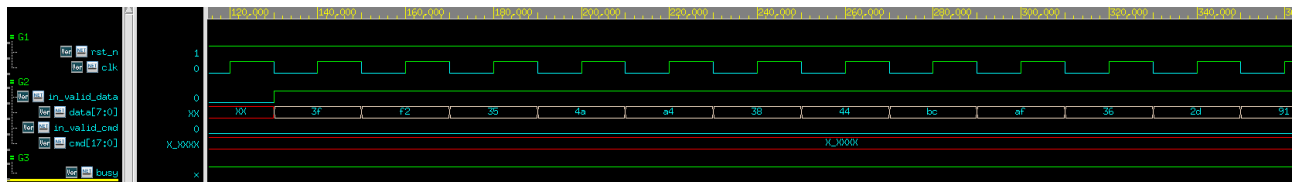
`./02_check`

Reference Waveform

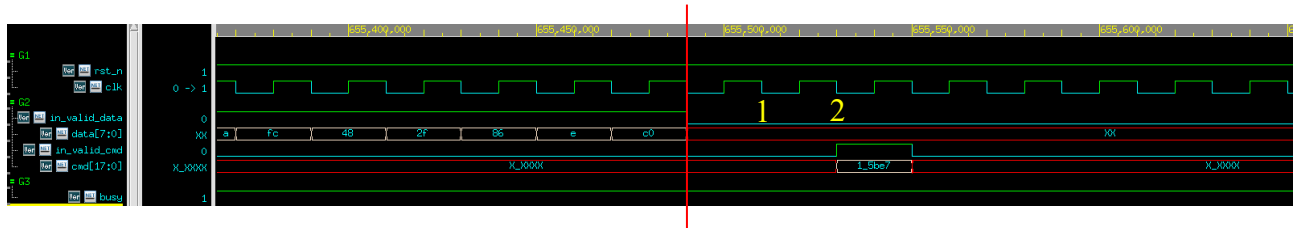
- Asynchronous reset and active-low reset all output to low



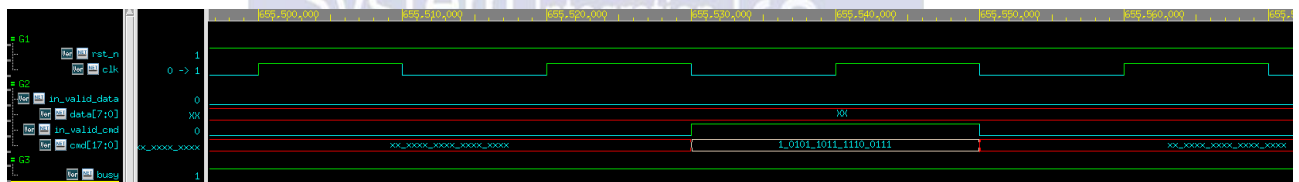
- Input of 8-bit data when in_valid_data is tied high



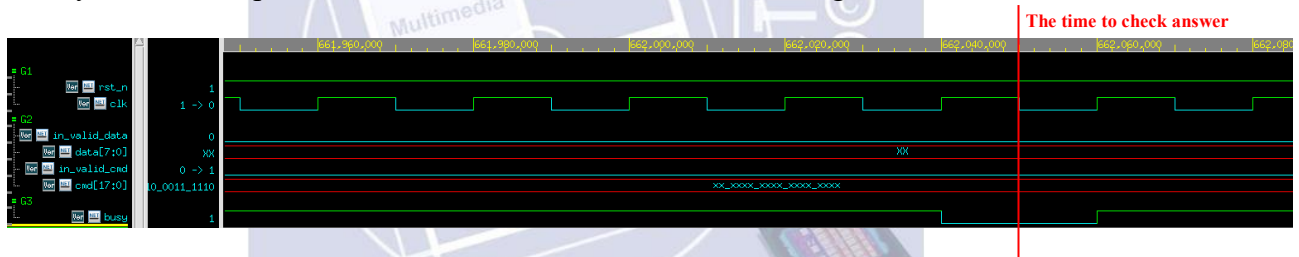
- in_valid_cmd will come in 2~4 negative edge of clock after in_valid_data is tied low



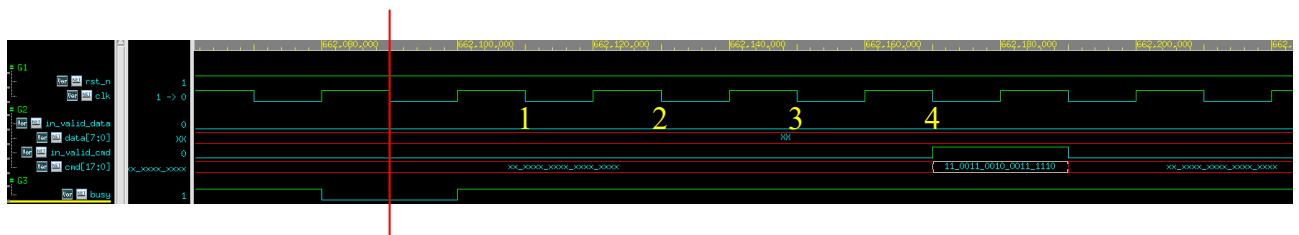
- Input of 18-bit cmd when in_valid_cmd is tied high for one cycle



- If GTE finish the operation based on the current command, busy should be tied low for at least one cycle and then pattern will check the data of destination image stored in SRAM.



- The next input pattern will come in 2~4 negative edge of clock after the busy falls and pattern checks answer



- The definition of latency

